Fundamentals of Machine Learning

Tutorial 1, Extra Practice: Problems with Solutions

From learning problems to probabilistic classification. Same skills as Tutorial 1, new settings and numbers.

How to use this set

Eight problems, one for each problem of Tutorial 1, in the same order and with the same skills: formulating a learning problem, decisions from costs, the geometry of KNN, auditing an evaluation, capacity and selection, discriminative probabilities, generative models, and detection over a sequence. The settings and the numbers are new, and each problem adds one twist the tutorial did not ask. Part B is ten two-minute drills for speed.

Each problem opens with a blue box holding the setup and a suggested time, then a yellow box listing the tools you need with a pointer [one-read §x] to the one-read document. Each sub-question is followed by its full solution. Difficulty: ⋆ direct application, ⋆⋆ several steps, ⋆⋆⋆ derivation or proof. All logarithms are natural.

Contents

1 Part A: Tutorial-style problems **1**

1.1 Problem 1: From a phishing objective to a learning problem 1

1.2 Problem 2: The optimal action depends on the loss 3

1.3 Problem 3: KNN inherits the geometry of the representation 5

1.4 Problem 4: Audit a generalisation claim 7

1.5 Problem 5: Capacity, regularisation and validation 8

1.6 Problem 6: Discriminative probabilities, decisions and prior shift 10

1.7 Problem 7: Generative classification, assumptions become visible 12

1.8 Problem 8: Detecting a brute-force attack from a sequence of logins 15

2 Part B: Quick drills **17**

1 Part A: Tutorial-style problems

1.1 Problem 1: From a phishing objective to a learning problem

Problem 1 (⋆) 15 minutes

A company's mail gateway receives every incoming email before delivery. The raw record contains the headers (sender and reply-to addresses, routing hops, SPF/DKIM/DMARC authentication results), the subject, the body, the links and the attachments. Labels come from triage: when users report an email, an analyst marks it *phishing* or *legitimate*. When an analyst confirms a campaign, every email sent from the same sending infrastructure is also labelled phishing, even if that particular email was never reported. The deployed system must score each incoming email and decide whether to *deliver* it, *deliver it with a warning banner*, or *quarantine* it.

Tools you need

The pipeline object → representation → target → model output → action [one-read §1.4, 1.5]; the posterior versus the decision [one-read §2.2]; leakage and shortcuts [one-read §3.6]; the three learning settings [one-read §1.2].

(a) Identify the physical object, the representation x, the target Y, the model output, and the final action.

- **Physical object:** the email message as it arrives at the gateway, together with its delivery context (time, sending server).
- **Representation x:** a vector computed from that message at arrival. Examples: the authentication results, the age of the sender domain, the number of links, whether the visible link text differs from the real link domain, the presence of urgent wording, the attachment type, word counts from the body.
- **Target $Y \in \{0, 1\}$:** 1 = phishing, 0 = legitimate, *as defined by the triage process*. Note that the label includes the campaign-level rule, so Y means "belongs to a confirmed phishing campaign" rather than "this text is deceptive".
- **Model output:** a score $s_\theta(x)$ or a posterior estimate $\hat{p}_\theta(y = 1 \mid x)$.
- **Final action:** deliver, deliver with banner, or quarantine. A decision rule chooses it from the posterior and the costs of each mistake.

(b) A legitimate password-reset email from the company's IT provider and a phishing email cloned from it can have almost identical observed features. Explain why a classifier using only these features may be unable to separate them. Does this mean the labels are wrong?

If two emails have the same x but different labels, then $p(y = 1 \mid x)$ is strictly between 0 and 1 for that x. Any rule that looks only at x gives the same answer to both, so one of them is misclassified: the Bayes error is positive [one-read §2.2]. No amount of data or model capacity fixes this, because the missing information is not in x.

The labels can still be perfectly correct. What separates the two emails is information the representation does not contain: who controls the sending server, where the link really leads, whether the email belongs to a confirmed campaign. The remedy is to add features that carry that information (sending-IP reputation, the resolved link destination), not to "clean" the labels.

(c) Explain why a posterior estimate $\hat{p}_\theta(y = 1 \mid x)$ and the quarantine decision are not the same object.

The posterior is a *belief* about the class. The decision also depends on things the posterior does not contain [one-read §2.2]:

- **The costs.** Quarantining an urgent legitimate invoice can cost more in the finance department than elsewhere, so the same $\hat{p} = 0.3$ can lead to different actions for different recipients.
- **The available actions.** The banner is a cheap middle action, and it changes the optimal regions.
- **The quality of the estimate.** $\hat{p}_\theta$ is an estimate and may be miscalibrated. The decision rule treats it as the true posterior, so calibration matters.

(d) For each candidate feature, say whether it is available at decision time and whether it would create target leakage: (i) the number of recipients who reported the email; (ii) whether the sender domain was registered less than 30 days before arrival; (iii) whether a link in the email was later added to a blocklist; (iv) the DKIM result.

feature	verdict
(i) number of user reports	**Leakage.** Reports arrive *after* delivery and are part of the labelling process itself.
(ii) domain younger than 30 days	**Usable**, provided the age is computed from registration data known at arrival time, not from a later lookup.
(iii) link later blocklisted	**Leakage.** It is future information, and it is closely tied to the analyst's verdict.
(iv) DKIM result	**Usable.** The gateway computes it on arrival.

The test for every feature: *could the deployed system compute this value at the moment it must decide, using only data available then?*

(e) In the collected data, the phishing emails were exported from a quarantine archive that converts every message to plain text, while the legitimate emails were exported from user mailboxes in their original HTML. Explain how "the email contains HTML formatting" can become a learning shortcut. Why is this different from target leakage? How could you detect the problem?

In the training data, "no HTML" almost perfectly predicts "phishing". A model can reach low training and test error by learning that rule alone, without learning anything about deception. At deployment every email arrives in its original format, phishing included, so the rule fails.

Shortcut versus leakage. The formatting feature *is* available at decision time, so this is not leakage. The problem is that its relation to the label is an artefact of how the data were collected and will not hold in deployment. Leakage uses information the system will not have when it decides; a shortcut uses information it will have, whose link to the label is spurious.

Detection. Compare each feature's distribution per class *and per collection source*; look for features that predict the label too well; retrain without the suspicious feature and see whether performance collapses; best of all, export both classes through the same pipeline.

(f) Describe one email-security task for each paradigm: supervised, self-supervised, and reinforcement learning. For each, identify the information that provides the learning signal.

- **Supervised:** classify emails as phishing or legitimate. Signal: the analyst labels.
- **Self-supervised:** pretrain a text model on millions of unlabelled emails by hiding words and predicting them, then reuse its representation for the classifier. Signal: the email text itself (the hidden part is the target).
- **Reinforcement learning:** an agent decides, email by email, whether to deliver, banner, or quarantine, and receives a reward from what follows (user complaints about blocked mail, incidents caused by delivered phishing). Signal: the delayed reward, not a label per email [one-read §1.2].

1.2 Problem 2: The optimal action depends on the loss

Problem 2 (⋆⋆) 15 minutes

An endpoint-protection agent must decide what to do with a newly downloaded file. For a file with features x let $p = p(y = 1 \mid x)$, where class 1 means *malicious*. Allowing a malicious file costs $C_{FN} = 12$. Blocking a benign file costs $C_{FP} = 3$, because it interrupts the user's work. Correct decisions cost 0.

Tools you need

Conditional risk of an action: the average loss given x, $R(a \mid x) = \sum_y c(a, y)\, p(y \mid x)$. Bayes rule: take the action with the smallest conditional risk [one-read §2.2].

(a) Derive the conditional risks of *allow* and *block*, and the optimal probability threshold.

$$\textbf{allow} \quad R(\text{allow} \mid x) = C_{FN}\, p = 12p,$$
$$\textbf{block} \quad R(\text{block} \mid x) = C_{FP}\,(1-p) = 3(1-p).$$

Block when $3(1-p) < 12p \iff 3 < 15p \iff p > 0.2$. In general the threshold is $t = C_{FP}/(C_{FP}+C_{FN}) = 3/15 = 0.2$.

(b) Apply the rule to four files with posteriors 0.05, 0.18, 0.25, 0.60.

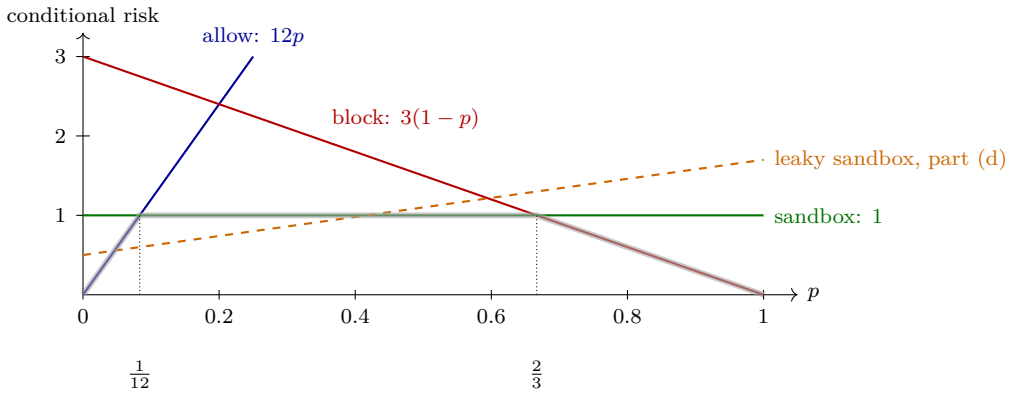
p	$R(\text{allow}) = 12p$	$R(\text{block}) = 3(1-p)$	decision
0.05	0.60	2.85	allow
0.18	2.16	2.46	allow
0.25	3.00	2.25	block
0.60	7.20	1.20	block

(c) A sandbox can now detonate the file and observe its behaviour. Sandboxing costs 1 whatever the true class, and then the file is handled correctly. Derive the probability regions for allow, sandbox, and block, and redo (b).

The new action has constant risk $R(\text{sandbox} \mid x) = 1$.

$$\textbf{allow optimal} \quad 12p \le 1 \text{ and } 12p \le 3(1-p) \iff p \le 1/12 \approx 0.083 \text{ (and } p \le 0.2),$$
$$\textbf{block optimal} \quad 3(1-p) \le 1 \text{ and } 3(1-p) \le 12p \iff p \ge 2/3 \text{ (and } p \ge 0.2),$$
$$\textbf{sandbox optimal} \quad 1/12 < p < 2/3.$$

The four files become: 0.05 allow, 0.18 *sandbox*, 0.25 *sandbox*, 0.60 *sandbox*. Only the very clear cases are decided without the sandbox.



The grey path is the lower envelope: the risk of the best action at each p.

(d) The sandbox is leaky. It costs only 0.5, and benign files are always cleared correctly, but 10% of malicious files detect the sandbox, behave innocently, and are then allowed. Derive the conditional risk of sandboxing, the new regions, and the decisions for the four files.

If the file is benign (probability $1-p$) the cost is 0.5. If it is malicious (probability p) the cost is 0.5, plus $C_{FN} = 12$ with probability 0.1:

$$R(\text{sandbox} \mid x) = 0.5 + p\,(0.1 \times 12) = 0.5 + 1.2p.$$

$$\begin{aligned}
\textbf{allow optimal} \quad & 12p \le 0.5 + 1.2p \iff 10.8p \le 0.5 \iff p \le 5/108 \approx 0.046, \\
\textbf{block optimal} \quad & 3(1-p) \le 0.5 + 1.2p \iff 2.5 \le 4.2p \iff p \ge 25/42 \approx 0.595, \\
\textbf{sandbox optimal} \quad & 0.046 < p < 0.595.
\end{aligned}$$

Decisions: 0.05 *sandbox* (risk 0.56 against 0.60 for allow), 0.18 sandbox, 0.25 sandbox, 0.60 *block* (risk 1.20 against 1.22). A cheaper but imperfect review widens the band at the bottom, because it is very cheap for nearly benign files, and narrows it at the top, because for likely-malicious files the 10% escape rate becomes expensive.

(e) Back to the perfect sandbox of (c), but with unknown cost c. What is the largest c for which sandboxing is optimal for at least one file? Interpret.

Without the sandbox the best risk is $\min\{12p,\ 3(1-p)\}$. This is largest where the two lines cross, at $p = 0.2$, with value $12 \times 0.2 = 2.4$. A constant line c dips below that envelope somewhere if and only if

$$c < 2.4.$$

So the sandbox is worth having only if it costs less than the risk of the hardest file, the one sitting exactly at the threshold. Above 2.4 it is never used, however uncertain the classifier is.

(f) On the build servers, blocking a benign file stops a release and costs 9 instead of 3. Compute the new threshold, and decide the file with $p = 0.25$ in both settings.

$t = 9/(9+12) = 3/7 \approx 0.43$. The file with $p = 0.25$ is *blocked* on a laptop ($0.25 > 0.2$) and *allowed* on a build server ($0.25 < 0.43$). Same model, same posterior, different action. That is part (c) of Problem 1 in numbers.

1.3 Problem 3: KNN inherits the geometry of the representation

Problem 3 (⋆⋆) **15 minutes**

A host is described by $x = (x_1, x_2)$, where x_1 is the number of distinct internal hosts it contacted in the last 10 minutes and x_2 the volume it uploaded to external addresses, in MB. A query host has

$$x = (5,\ 40).$$

Two labelled hosts are $x_A = (5,\ 200)$, labelled *attack* (data exfiltration), and $x_B = (20,\ 50)$, labelled *benign* (a backup server).

Tools you need

Euclidean distance, rescaling $z_j = x_j / c_j$; for 1-NN with two training points, the boundary is the set of points equidistant from both, the perpendicular bisector [one-read §2.1, 2.4].

(a) Using raw Euclidean distance, determine the nearest host.

$$\begin{aligned}
d(x, x_A) &= \sqrt{(5-5)^2 + (40-200)^2} = \sqrt{25600} = 160, \\
d(x, x_B) &= \sqrt{(5-20)^2 + (40-50)^2} = \sqrt{225+100} = \sqrt{325} \approx 18.0.
\end{aligned}$$

Nearest: **B** (benign). The upload volume is in the hundreds of MB, so it dominates the distance; a difference of 15 hosts barely counts.

(b) Scale the host count by 5 hosts and the upload by 100 MB. Determine the nearest host after scaling.

$z = (5/5,\ 40/100) = (1,\ 0.4)$, $z_A = (1,\ 2)$, $z_B = (4,\ 0.5)$.

$$d^2(z, z_A) = 0^2 + (0.4 - 2)^2 = 2.56, \qquad d = 1.60,$$
$$d^2(z, z_B) = (1 - 4)^2 + (0.4 - 0.5)^2 = 9 + 0.01 = 9.01, \qquad d \approx 3.00.$$

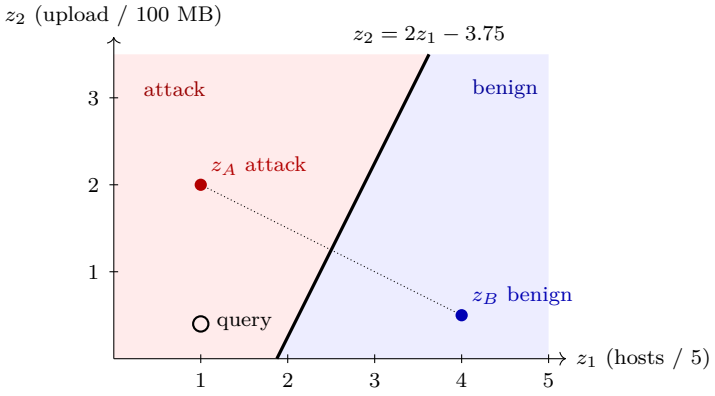
Nearest: **A** (attack). In Tutorial 1 the flip went the other way: here, scaling gives the host count a voice, and the query contacts exactly as many hosts as the exfiltrating machine.

(c) In the scaled (z_1, z_2) plane, derive and plot the 1-NN decision boundary between z_A and z_B. Mark the query and give its predicted class.

The boundary is the set of z with $\|z - z_A\|^2 = \|z - z_B\|^2$. Expanding, the $\|z\|^2$ terms cancel:

$$2(z_B - z_A)^\top z = \|z_B\|^2 - \|z_A\|^2 \iff 2(3, -1.5)^\top z = (16 + 0.25) - (1 + 4) \iff 6z_1 - 3z_2 = 11.25,$$

that is, $z_2 = 2z_1 - 3.75$. It passes through the midpoint $(2.5,\ 1.25)$ and is perpendicular to $z_B - z_A = (3, -1.5)$. The attack side is where $2z_1 - z_2 < 3.75$: check z_A gives 0. At the query, $2(1) - 0.4 = 1.6 < 3.75$, so the query is **attack**.



(d) Use the weighted squared distance $d_w^2(z, z') = w(z_1 - z_1')^2 + (z_2 - z_2')^2$ with $w > 0$. Derive the critical w at which the nearest host changes, say which host is nearest on each side, and interpret.

$$d_w^2(z, z_A) = w \cdot 0 + 2.56 = 2.56, \qquad d_w^2(z, z_B) = 9w + 0.01.$$

They are equal when $9w + 0.01 = 2.56$, i.e. $w^\star = 2.55/9 = 17/60 \approx 0.283$.

- $w < 17/60$: B is nearest (benign). The host count matters little, and the uploads 40 versus 50 MB look alike.
- $w > 17/60$: A is nearest (attack). The host count matters, and the query matches A exactly on it.

The prediction is decided by a modelling choice, not by the data. That is why w is a hyperparameter to be chosen on validation data, never on the test set.

(e) A colleague adds a third feature, the destination port as a raw integer (22, 443, 3389, 8080). Why is Euclidean distance on this feature meaningless? Propose a better representation.

A port number is a *category*, not a quantity. Under Euclidean distance, port 443 (HTTPS) is "closer" to 22 (SSH) than to 3389 (remote desktop), and 8080 is far from everything. These distances reflect the arbitrary numbering, not similarity of behaviour. Worse, the raw values are in the thousands, so this feature would dominate the distance, exactly as in (a).

Better: a one-hot encoding of a small set of service groups (web, remote administration, file sharing, other), or indicators such as "remote-administration port". In a one-hot encoding every two different categories are at the same distance.

(f) The scales 5 and 100 were obtained as the standard deviations of the two features over the whole dataset, test set included. Is this leakage? What if they had been chosen from domain knowledge?

Computing scaling statistics on data that includes the test set lets information about the test set shape the model. That is leakage, usually mild for a standard deviation but leakage nonetheless; the scaler must be fitted on the training data only, and refitted inside each cross-validation fold [one-read §3.6]. Fixed scales chosen from domain knowledge before seeing the data ("5 hosts" and "100 MB" as meaningful units) use no data at all, so they cause no leakage.

1.4 Problem 4: Audit a generalisation claim

Problem 4 (⋆⋆) 15 minutes

A team collects 60,000 Windows executables gathered between 2019 and 2025: malware from many families, many of them repacked variants of one another, and benign software including many versions of the same programs. The team proceeds as follows:

- fill missing values with the median computed on the full dataset;
- keep the 200 features most correlated with the label, computed on the full dataset;
- randomly split individual files 80/20 into training and test sets;
- tune hyperparameters by 5-fold cross-validation on the training set, with random folds;
- inspect the misclassified test files, add features that fix them, retrain; repeat 15 times;
- report the final test accuracy as the expected performance on next year's malware.

Tools you need

Three subsets and their roles [one-read §2.7]; selection and leakage [one-read §3.5, 3.6]; precision, recall, F_1, and the base rate [one-read §3.6].

(a) Identify every mechanism that can make the reported performance too optimistic.

1. **Preprocessing on the full dataset.** The medians and, much more seriously, the feature selection have seen the test labels. Choosing the 200 features most correlated with the label on all the data selects features that happen to fit the test files.
2. **Near duplicates across the split.** Repacked variants of one family, and versions of one program, land on both sides of a random file-level split. The test set then measures recognition of near-copies, not generalisation.
3. **The same problem inside cross-validation.** Random folds put variants of one family in both training and validation folds, so hyperparameters are tuned for memorisation.
4. **Test-set reuse.** Fifteen rounds of "look at the test errors, fix them" turn the test set into a training signal. After that it is no longer an independent estimate [one-read §3.5].
5. **Time travel.** A random split trains on 2025 malware and tests on 2019 malware. That is the opposite of deployment, and it hides how fast malware changes.
6. **The claim itself.** Accuracy on a random split of 2019–2025 data says nothing directly about next year's malware.

(b) Propose a training, validation and test protocol for each goal: (i) new files from the same distribution; (ii) next year's malware; (iii) malware families never seen before.

In every case, fit all preprocessing (imputation, selection, scaling) inside the training part only, and touch the test set once at the very end.

(i) Split by *group*, not by file: put all variants of one malware family, and all versions of one benign program, on the same side. Use grouped cross-validation on the training part for tuning.

(ii) Split by *time*: train on 2019–2023, validate on 2024, test on 2025. Retrain on everything up to

the test period only after model choices are fixed. This measures performance on the future.

(iii) Split by *family*: hold out whole families for validation and for testing (leave-families-out). Use only families never seen in training to measure detection of new families.

(c) A test set of 20,000 files contains 400 malicious ones. The classifier gives $TP = 300$, $FP = 600$, $FN = 100$, $TN = 19{,}000$. Compute accuracy, precision, recall, F_1, and the false-positive rate.

$$\text{accuracy} = \frac{300 + 19000}{20000} = 0.965, \qquad \text{precision} = \frac{300}{300 + 600} = \frac{1}{3} \approx 0.333,$$

$$\text{recall} = \frac{300}{300 + 100} = 0.75, \qquad F_1 = \frac{2 \times \frac{1}{3} \times \frac{3}{4}}{\frac{1}{3} + \frac{3}{4}} = \frac{1/2}{13/12} = \frac{6}{13} \approx 0.462,$$

$$\text{FPR} = \frac{600}{600 + 19000} = \frac{3}{98} \approx 0.031.$$

(d) Compare with the rule that declares every file benign. Why is accuracy insufficient here?

The all-benign rule has accuracy $19600/20000 = 0.98$, *higher* than the classifier's 0.965. It catches no malware at all: recall 0, and precision undefined. Accuracy is dominated by the 98% benign majority, so it rewards doing nothing. With rare positives, report precision and recall (or F_1, or a cost-weighted loss), and compare them with what the trivial rule achieves.

(e) At deployment only 1 file in 200 is malicious. Assuming the same recall and false-positive rate, compute the precision. What fraction of alerts are false?

Write $\pi = 0.005$. Out of N files, true alerts number $0.75\,\pi N$ and false alerts $\frac{3}{98}(1 - \pi)N$:

$$\text{precision} = \frac{0.75 \times 0.005}{0.75 \times 0.005 + \frac{3}{98} \times 0.995} = \frac{0.00375}{0.00375 + 0.03046} \approx 0.110.$$

About 89% of alerts are false, compared with 67% on the test set. Recall and false-positive rate do not depend on the base rate; precision does, and it falls quickly as positives become rarer.

(f) State where imputation, feature selection and scaling must be fitted in a cross-validation pipeline.

Inside each fold, on the training part of that fold only: fit the imputer's medians, the feature selector, and the scaler on the fold's training data, then *apply* them unchanged to the fold's validation data. Wrap everything in one pipeline object so that cross-validation refits all steps every time. After model selection, refit the whole pipeline on the full training set and apply it once to the test set.

1.5 Problem 5: Capacity, regularisation and validation

Problem 5 ($\star\star\star$) **20 minutes**

A logistic regression is trained by minimising $J_\lambda(\theta) = L(\theta) + \lambda\|\theta\|^2$, where L is the average training cross-entropy. Let $\hat{\theta}_\lambda$ be a minimiser. For five values of λ the team records the training loss $L(\hat{\theta}_\lambda)$ (without the penalty) and the validation error rate:

λ	10	1	0.1	0.01	0
training loss $L(\hat{\theta}_\lambda)$	0.52	0.38	0.29	0.24	0.21
validation error	0.27	0.19	0.15	0.17	0.23

Tools you need

Optimality: $J_\lambda(\hat{\theta}_\lambda) \le J_\lambda(\theta)$ for every θ. Capacity, underfitting and overfitting [one-read §3.2]; regularisation [one-read §3.4]; selection noise [one-read §3.5]. For $X \sim \mathcal{N}(\mu, s^2)$: $\mathbb{E}|X| = s\sqrt{2/\pi}\, e^{-\mu^2/(2s^2)} + \mu\,(1 - 2\Phi(-\mu/s))$. Values: $\Phi(0.7071) = 0.7602$, $e^{-0.25} = 0.7788$, $\sqrt{2/\pi} = 0.7979$.

(a) Let $\lambda_1 > \lambda_2 \ge 0$, and write $\theta_1 = \hat{\theta}_{\lambda_1}$, $\theta_2 = \hat{\theta}_{\lambda_2}$. Prove that $\|\theta_1\|^2 \le \|\theta_2\|^2$ and $L(\theta_2) \le L(\theta_1)$: weaker regularisation can never increase the training loss.

Each minimiser is at least as good as the other one for its own objective:

$$L(\theta_1) + \lambda_1\|\theta_1\|^2 \le L(\theta_2) + \lambda_1\|\theta_2\|^2, \tag{1}$$

$$L(\theta_2) + \lambda_2\|\theta_2\|^2 \le L(\theta_1) + \lambda_2\|\theta_1\|^2. \tag{2}$$

Norms. Add (1) and (2). The L terms cancel and we are left with

$$\lambda_1\|\theta_1\|^2 + \lambda_2\|\theta_2\|^2 \le \lambda_1\|\theta_2\|^2 + \lambda_2\|\theta_1\|^2 \iff (\lambda_1 - \lambda_2)(\|\theta_1\|^2 - \|\theta_2\|^2) \le 0.$$

Since $\lambda_1 - \lambda_2 > 0$, this gives $\|\theta_1\|^2 \le \|\theta_2\|^2$.

Losses. Rearrange (2): $L(\theta_2) \le L(\theta_1) + \lambda_2(\|\theta_1\|^2 - \|\theta_2\|^2)$. The bracket is ≤ 0 by the first step and $\lambda_2 \ge 0$, so $L(\theta_2) \le L(\theta_1)$. ■

This is the regularised version of the tutorial's nested-families argument: lowering λ enlarges the effective model, so the fit to the training data can only improve. The table agrees: the training loss falls monotonically from 0.52 to 0.21.

(b) Does (a) guarantee that the training *error rate* also decreases as λ decreases? Explain.

No. The proof only uses the fact that $\hat{\theta}_\lambda$ minimises J_λ, which contains the cross-entropy L. The error rate is a different criterion. A model can lower its cross-entropy by becoming more confident on points it already classifies correctly, while flipping one borderline point to the wrong side. Usually the two move together, but only the minimised criterion is guaranteed to be monotone.

(c) Which λ would you select? Diagnose the others.

Select $\lambda = 0.1$, which has the lowest validation error (0.15).

- $\lambda = 10$ and $\lambda = 1$: high training loss and high validation error. Too much regularisation, so the model *underfits* (high bias).
- $\lambda = 0.01$ and $\lambda = 0$: the training loss keeps falling but the validation error rises again. Too little regularisation, so the model fits noise in the sample (high estimation variance, *overfitting*). The growing gap between training and validation performance is the signature.

(d) The data grow from 2,000 to 200,000 training examples. Should the team keep $\lambda = 0.1$? Explain.

Not without re-validating. Overfitting comes from the variance of the estimate, which shrinks as the sample grows. With 100 times more data the same model can afford more freedom, so the best λ typically *decreases*. Capacity is only too large or too small relative to the sample size, so λ must be re-selected on validation data for the new sample.

(e) Two configurations have population errors $R_1 = 0.20$ and $R_2 = 0.22$. Their validation errors are $\widehat{R}_k = R_k + \varepsilon_k$ with $\varepsilon_1, \varepsilon_2$ independent $\mathcal{N}(0, \sigma^2)$ and $\sigma = 0.02$. The configuration with the smaller validation error is selected. Compute (i) the probability of selecting the worse configuration, (ii) the expected reported validation error, and (iii) the expected population error of the selected configuration. Interpret.

Let $D = \widehat{R}_1 - \widehat{R}_2 = (R_1 - R_2) + (\varepsilon_1 - \varepsilon_2) \sim \mathcal{N}(-0.02,\ s^2)$ with $s = \sigma\sqrt{2} \approx 0.02828$.

(i) The worse configuration (2) is chosen when $D > 0$:

$$P(D > 0) = P\Big(\frac{D + 0.02}{s} > \frac{0.02}{s}\Big) = 1 - \Phi(0.7071) = 0.2398.$$

(ii) With $\min(a, b) = \frac{a+b-|a-b|}{2}$, $\mathbb{E}[\min(\widehat{R}_1, \widehat{R}_2)] = \frac{0.20+0.22-\mathbb{E}|D|}{2}$. Apply the folded-normal formula with $\mu = -0.02$, $\mu/s = -0.7071$, $\mu^2/(2s^2) = 0.25$:

$$\mathbb{E}|D| = 0.02828 \times 0.7979 \times 0.7788 + (-0.02)(1 - 2\Phi(0.7071)) = 0.01758 + (-0.02)(-0.5205) = 0.02799,$$

so $\mathbb{E}[\text{reported}] = (0.42 - 0.02799)/2 \approx 0.1960$.

(iii) $\mathbb{E}[R_{\text{selected}}] = 0.20 \times 0.7602 + 0.22 \times 0.2398 \approx 0.2048$.

Interpretation. The selected model's reported error (0.196) is *below* even the best population error (0.20), and below the error we actually get (0.205). Selection picks whichever configuration was lucky on this validation set, so the winning score is biased downward. One time in four we even pick the worse model. Hence the selected model must be scored again on data that played no part in the choice.

(f) The team now tries 100 configurations, all with population error 0.20 and the same σ. Given that the expected maximum of 100 independent standard normals is about 2.51, estimate the expected reported validation error of the winner.

$\min_k(R + \sigma Z_k) = R - \sigma \max_k(-Z_k)$, and $-Z_k$ are also standard normals, so $\mathbb{E}[\text{reported}] \approx 0.20 - 2.51 \times 0.02 \approx 0.150$. Every configuration is equally good, yet the winner reports a 15% error against a true 20%. This is exactly what step 5 of Problem 4 did with its test set.

1.6 Problem 6: Discriminative probabilities, decisions and prior shift

Problem 6 (⋆⋆) **20 minutes**

A login service scores each login attempt for account takeover. A logistic model produces a score $s_\theta(x)$ and the posterior estimate

$$r_t(x) = \hat{p}_\theta(y = 1 \mid x) = \sigma(s_\theta(x)) = \frac{1}{1 + e^{-s_\theta(x)}},$$

where class 1 means *takeover*. The available actions are *allow* and *challenge* (ask for a second factor).

Tools you need

Sigmoid and logit, $\text{logit}(p) = \ln\frac{p}{1-p}$; cross-entropy $\ell = -[y \ln r + (1 - y)\ln(1 - r)]$ [one-read §4.2]; cost threshold [one-read §2.2]; prior-shift correction [one-read §4.7]:

$$r_d(x) = \frac{(\pi_d/\pi_t)\, r_t(x)}{(\pi_d/\pi_t)\, r_t(x) + \frac{1-\pi_d}{1-\pi_t}\,[1 - r_t(x)]}.$$

(a) For $s_\theta(x) = -0.4$, compute the posterior estimate, and the cross-entropy loss if the true label is 0 and if it is 1.

$r = 1/(1 + e^{0.4}) = 1/(1 + 1.4918) \approx 0.4013$.

$$y = 0:\ \ell = -\ln(1 - 0.4013) = -\ln 0.5987 \approx 0.513, \qquad y = 1:\ \ell = -\ln 0.4013 \approx 0.913.$$

The difference is exactly 0.4, the score. Indeed $\ln\frac{1-r}{r} = -s$, so the loss gap between the two labels always equals $|s|$.

(b) Show that minimising the empirical cross-entropy is equivalent to maximising the conditional likelihood of the labels under an i.i.d. assumption. Then show that $\partial \ell / \partial s = r - y$, and interpret the sign.

Under the model, $Y_i \mid x_i \sim \text{Bernoulli}(r_i)$ with $r_i = \sigma(s_\theta(x_i))$, so $P(y_i \mid x_i) = r_i^{y_i}(1-r_i)^{1-y_i}$. With independent examples the conditional likelihood is a product, and its log is a sum:

$$\ln \prod_i r_i^{y_i}(1-r_i)^{1-y_i} = \sum_i \left[y_i \ln r_i + (1-y_i)\ln(1-r_i) \right] = -\sum_i \ell_i.$$

Maximising the log-likelihood is minimising the summed (or averaged) cross-entropy; ln is increasing, so the maximisers coincide.

Derivative. $\sigma'(s) = \sigma(s)(1-\sigma(s)) = r(1-r)$. Hence

$$\frac{\partial \ell}{\partial s} = -\Big[\frac{y}{r} - \frac{1-y}{1-r}\Big] r(1-r) = -\big[y(1-r) - (1-y)r\big] = r - y.$$

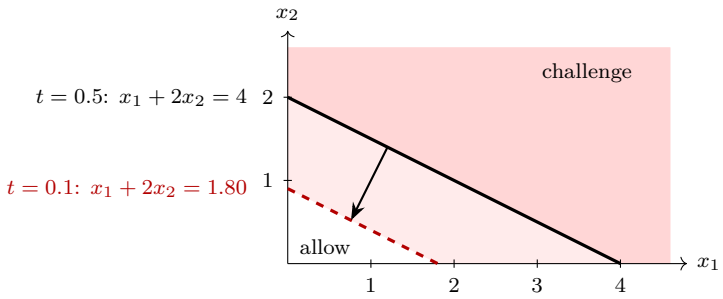
Sign. If $y = 1$ the derivative is $r - 1 < 0$, so gradient descent raises the score. If $y = 0$ it is $r > 0$, so the score is lowered. The size of the push is the error $|r - y|$: confident correct predictions barely move.

(c) A needless challenge costs $C_{FP} = 1$ and a missed takeover costs $C_{FN} = 9$. Determine the threshold on the posterior and on the score, and the decision for the login of (a).

$t = 1/(1+9) = 0.1$. On the score: $r \geq 0.1 \iff s \geq \text{logit}(0.1) = \ln(1/9) \approx -2.197$. The login of (a) has $r \approx 0.40 \geq 0.1$ (equivalently $s = -0.4 \geq -2.197$), so: **challenge**.

(d) For the two-feature model $s_\theta(x) = -4 + x_1 + 2x_2$, plot the decision boundary for threshold 0.5 and for the threshold of (c). Indicate the challenge region and explain the movement.

Threshold 0.5 means $s = 0$: $x_1 + 2x_2 = 4$. Threshold 0.1 means $s = -2.197$: $x_1 + 2x_2 = 4 - 2.197 = 1.803$. Both lines have the same normal $(1, 2)$, so they are parallel; the challenge region is the side where $x_1 + 2x_2$ is large.



A lower threshold moves the boundary parallel to itself, towards the low-score side. The shift is $2.197/\|(1,2)\| = 2.197/\sqrt{5} \approx 0.98$ units. The classifier is unchanged; only the action changes. The challenge region grows because misses are nine times as costly as needless challenges.

(e) The model was trained on a balanced sample ($\pi_t = 0.5$) obtained by discarding most legitimate logins, while in deployment $\pi_d = 0.02$. Assuming prior shift and a calibrated $r_t(x) = 0.9$, compute $r_d(x)$.

$\pi_d/\pi_t = 0.04$ and $(1-\pi_d)/(1-\pi_t) = 0.98/0.5 = 1.96$:

$$r_d = \frac{0.04 \times 0.9}{0.04 \times 0.9 + 1.96 \times 0.1} = \frac{0.036}{0.036 + 0.196} = \frac{0.036}{0.232} \approx 0.155.$$

A login that looks 90% suspicious in the balanced world is only about 16% likely to be a takeover in the real one, where takeovers are 25 times rarer.

(f) Show that the correction is a constant shift of the score, $s_d = s_t + \ln\frac{\pi_d}{1-\pi_d} - \ln\frac{\pi_t}{1-\pi_t}$. Compute the shift and check (e).

Divide the numerator and denominator of r_d by the second term of the denominator. The posterior odds are

$$\frac{r_d}{1-r_d} = \frac{\pi_d/\pi_t}{(1-\pi_d)/(1-\pi_t)} \cdot \frac{r_t}{1-r_t} = \frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} \cdot \frac{r_t}{1-r_t}.$$

Take logs. The log-odds is the score, so $s_d = s_t + \ln\frac{\pi_d}{1-\pi_d} - \ln\frac{\pi_t}{1-\pi_t}$. Here the shift is $\ln(0.02/0.98) - \ln 1 = -3.892$.

Check: $s_t = \text{logit}(0.9) = \ln 9 = 2.197$, so $s_d = 2.197 - 3.892 = -1.695$ and $\sigma(-1.695) \approx 0.155$, as in (e). Prior shift changes only the intercept of a logistic model.

(g) Combine (c) and (f): the rule "challenge if $r_d \geq 0.1$" is a threshold on the training posterior r_t. Find it. A colleague instead challenges when $r_t \geq 0.5$. Which logins are handled differently, and what does this cost?

$r_d \geq 0.1 \iff s_d \geq -2.197 \iff s_t \geq -2.197 + 3.892 = 1.695 \iff r_t \geq \sigma(1.695) \approx 0.845.$

The colleague's rule challenges every login with $0.5 \leq r_t < 0.845$, which the correct rule allows. Those logins have deployment posteriors between about 0.02 and 0.1, where a challenge is not worth its cost. The colleague's rule is too aggressive: it treats the balanced training world as reality, producing many needless challenges for real users. The two corrections (prior for estimation, costs for decision) push in opposite directions here, and both must be applied.

(h) Give one advantage and one drawback of having trained on a balanced sample built by discarding most legitimate logins.

Advantage: the takeover class is no longer drowned out. Its examples carry real weight in the loss, and training is cheaper. **Drawback:** the discarded legitimate logins carried information about the variety of normal behaviour, so the legitimate region is estimated less precisely; and the posteriors are biased towards takeover and must be corrected as in (e)–(f) before any cost-based decision. Weighting the legitimate examples instead of discarding them keeps the information while rebalancing the loss.

1.7 Problem 7: Generative classification, assumptions become visible

Problem 7, Part I: Gaussian class models (⋆⋆) 20 minutes for both parts

Two standardised features $x = (x_1, x_2)^\top$ are modelled as $p(x \mid y = c) = \mathcal{N}(\mu_c, \Sigma)$ with a shared covariance, where

$$\mu_0 = (0,0)^\top, \qquad \mu_1 = (3,0)^\top, \qquad \Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \pi_0 = 0.8, \quad \pi_1 = 0.2.$$

Tools you need

LDA score $s_c(x) = x^\top\Sigma^{-1}\mu_c - \frac{1}{2}\mu_c^\top\Sigma^{-1}\mu_c + \ln\pi_c$; posterior by normalising e^{s_0} and e^{s_1} [one-read §4.4]; the 2×2 inverse $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$.

(a) Compute Σ^{-1}, then $s_0(x)$ and $s_1(x)$.

$\det\Sigma = 4 - 1 = 3$, so $\Sigma^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$.

Class 0 has $\mu_0 = 0$, so $s_0(x) = \ln 0.8$. For class 1, $\Sigma^{-1}\mu_1 = \frac{1}{3}(6,-3)^\top = (2,-1)^\top$ and $\mu_1^\top\Sigma^{-1}\mu_1 =$

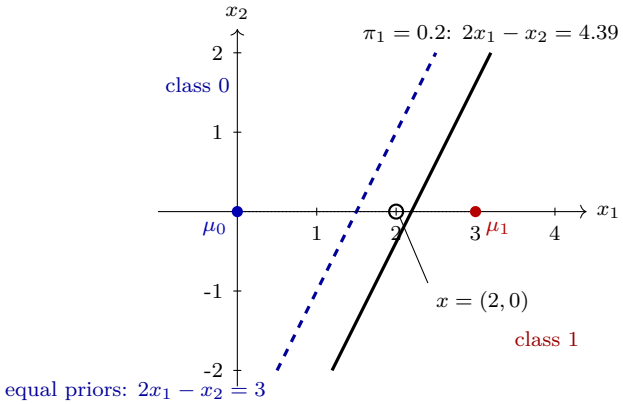
$(3,0)\cdot(2,-1)=6$:

$$s_1(x) = 2x_1 - x_2 - 3 + \ln 0.2.$$

(b) Write the boundary $s_0(x) = s_1(x)$ and plot it with the two means. Is it perpendicular to the segment joining the means? Where does it cross that segment?

$2x_1 - x_2 - 3 + \ln 0.2 = \ln 0.8 \iff 2x_1 - x_2 = 3 + \ln 4 \approx 4.386$.

Its normal vector is $\Sigma^{-1}(\mu_1 - \mu_0) = (2,-1)$, which is *not* parallel to $\mu_1 - \mu_0 = (3,0)$, so the boundary is **not perpendicular** to the segment. The correlation in Σ tilts it: since the features vary together along $(1,1)$, a move along that diagonal is "cheap" under the model and carries less evidence. On the segment $x = (x_1, 0)$ the boundary is at $x_1 = 4.386/2 \approx 2.19$, beyond the midpoint 1.5, because the prior favours class 0.



(c) For $x = (2,0)^\top$, determine the predicted class and the posterior probability of class 1.

$s_1 - s_0 = (4 - 0 - 3) + \ln 0.2 - \ln 0.8 = 1 - \ln 4 = -0.386$. The posterior is

$$P(y=1 \mid x) = \frac{e^{s_1}}{e^{s_0} + e^{s_1}} = \sigma(s_1 - s_0) = \sigma(-0.386) \approx 0.405,$$

so the predicted class is **0**.

(d) Repeat (b) and (c) with equal priors. Explain how the prior moves the boundary while the class models stay the same.

With $\pi_0 = \pi_1$ the $\ln \pi_c$ terms cancel: the boundary is $2x_1 - x_2 = 3$, which passes through the midpoint $(1.5, 0)$ (dashed line above). For $x = (2,0)$: $s_1 - s_0 = 1$, so $P(y=1 \mid x) = \sigma(1) \approx 0.731$ and the prediction flips to **class 1**.

The prior enters only through the constant $\ln(\pi_1/\pi_0)$, so changing it translates the boundary parallel to itself, away from the rarer class. The Gaussians, and therefore the direction of the boundary, do not change.

(e) What would change if each class had its own covariance matrix?

The quadratic terms $-\frac{1}{2}x^\top \Sigma_c^{-1} x$ and $-\frac{1}{2}\ln\det\Sigma_c$ no longer cancel between the classes, so the boundary becomes quadratic (QDA) [one-read §4.4]. The model gains flexibility but needs a covariance estimate per class: three parameters each in two dimensions, $d(d+1)/2$ in general, which is costly when one class is rare.

Problem 7, Part II: Naive Bayes from counts (⋆⋆)

A training set holds 200 emails: 40 phishing and 160 legitimate. Three binary features: $x_1 = 1$ if some link points to a raw IP address, $x_2 = 1$ if the text contains urgent wording, $x_3 = 1$ if the

sender is an internal company address. The number of emails with each feature equal to 1 is:

	x_1 (IP link)	x_2 (urgent)	x_3 (internal sender)
phishing (40)	18	30	0
legitimate (160)	4	32	64

A new email has $x = (1, 1, 1)$: an IP link and urgent wording, sent from an internal account.

Tools you need

Naive Bayes score $\pi_c \prod_j p(x_j \mid c)$, posterior by normalising [one-read §4.5]; Laplace smoothing for a binary feature: $\hat{p}(x_j = 1 \mid c) = (n_{jc} + 1)/(n_c + 2)$.

(a) Using the maximum-likelihood (unsmoothed) estimates, compute the posterior probability of phishing. Why is this dangerous here?

The MLE estimates are the relative frequencies. For phishing: $18/40 = 0.45$, $30/40 = 0.75$, $0/40 = 0$. For legitimate: $4/160 = 0.025$, $32/160 = 0.2$, $64/160 = 0.4$. The phishing score contains the factor $\hat{p}(x_3 = 1 \mid \text{phishing}) = 0$, so it is 0 and the posterior is **exactly 0**, whatever the other features say.

This is dangerous because phishing from a *compromised internal account* is a real and serious attack. It simply never appeared in 40 training emails. A single zero count vetoes all the other evidence, here an IP link that is 18 times more frequent in phishing than in legitimate email.

(b) Apply Laplace smoothing. Compute the two unnormalised scores and the posterior. With the cost threshold 0.1 from Problem 6, what is the decision?

Phishing ($n = 40$, denominators 42): $\frac{19}{42} = 0.4524$, $\frac{31}{42} = 0.7381$, $\frac{1}{42} = 0.0238$. Legitimate ($n = 160$, denominators 162): $\frac{5}{162} = 0.0309$, $\frac{33}{162} = 0.2037$, $\frac{65}{162} = 0.4012$.

$$\textbf{phishing} \quad 0.2 \times 0.4524 \times 0.7381 \times 0.0238 \approx 0.001590,$$

$$\textbf{legitimate} \quad 0.8 \times 0.0309 \times 0.2037 \times 0.4012 \approx 0.002018,$$

$$P(\text{phishing} \mid x) = \frac{0.001590}{0.001590 + 0.002018} \approx 0.441.$$

Below 0.5, but above the cost threshold 0.1: **quarantine** (or at least flag). Smoothing turned an absolute veto into strong but finite evidence against, which the other features can outweigh.

(c) Rewrite the computation in odds form, as the prior odds times one likelihood ratio per feature. Which features push towards phishing, and how hard?

$$\frac{P(\text{ph} \mid x)}{P(\text{leg} \mid x)} = \underbrace{\frac{0.2}{0.8}}_{0.25} \times \underbrace{\frac{0.4524}{0.0309}}_{14.66} \times \underbrace{\frac{0.7381}{0.2037}}_{3.62} \times \underbrace{\frac{0.0238}{0.4012}}_{0.059} \approx 0.788,$$

and $0.788/1.788 \approx 0.441$, as in (b). The IP link multiplies the odds by about 15 and urgency by about 3.6, while the internal sender divides them by about 17. In Naive Bayes each feature contributes one independent multiplicative vote; on the log scale the votes add.

(d) Suppose the feature set also contained x_2', "urgent wording in the subject", which in practice almost always equals x_2, with the same estimated probabilities. Recompute the posterior and explain what went wrong.

The odds are multiplied by the urgency ratio a second time: $0.788 \times 3.62 \approx 2.85$, so the posterior becomes $2.85/3.85 \approx 0.74$. One piece of evidence has been counted twice. The conditional-independence assumption says x_2 and x_2' give separate information given the class, but they are nearly the same measurement. Correlated features make Naive Bayes *overconfident*, pushing posteriors towards 0 or 1.

(e) State the Naive Bayes assumption used, and give one reason why it fails for these email features.

Given the class, the features are independent: $p(x_1, x_2, x_3 \mid c) = \prod_j p(x_j \mid c)$. It fails because the features are produced together by the same campaign: phishing kits that use IP links also tend to use urgent templates, and internal senders rarely use IP links. Within a class, the features are therefore correlated.

(f) Give one reason to prefer the discriminative route and one reason to prefer the generative route here.

Discriminative (logistic regression): it models $p(y \mid x)$ directly and does not assume independence, so correlated features such as x_2 and x_2' share their weight instead of double-counting, and the probabilities are usually better calibrated. **Generative** (Naive Bayes): it needs very little data per parameter, which matters with only 40 phishing emails; it handles missing features naturally; and its explicit $p(x \mid c)$ shows which feature values were never seen, as part (a) revealed.

1.8 Problem 8: Detecting a brute-force attack from a sequence of logins

Problem 8 (⋆⋆) 20 minutes

A detector observes SSH login attempts from one source address against a server. Attempt 1 is a legitimate login; a brute-force episode starts at $a_2 = 12$ s. Each row gives the start time a_i, the completion time t_i, the username tried, the outcome, and a per-attempt score s_i from a previously fitted classifier, available at t_i:

i	1	2	3	4	5	6	7
a_i (s)	0	12	20	27	35	41	50
t_i (s)	1	13	21	28	36	42	51
username	alice	root	admin	test	oracle	root	ubuntu
outcome	success	fail	fail	fail	fail	fail	fail
s_i	0.05	0.20	0.25	0.30	0.22	0.35	0.28

The per-attempt detector alerts when $s_i \geq 0.70$. A second detector, at each t_i, forms a window of length $W = 30$ s ending at t_i: it contains attempt i and every earlier attempt from the same source completed after $t_i - W$. It alerts when the window has *at least 4 attempts*, *at least 4 distinct usernames*, a *failure fraction of at least 0.75*, and a *score sum of at least 0.90*.

Tools you need

Window features computed only from the past; detection delay; grouping keys and leakage-safe splits for sequences [one-read §4.8, 3.6].

(a) Which attempts trigger the per-attempt detector? What does this show?

None: the largest score is $0.35 < 0.70$. Each failed login on its own looks only mildly suspicious, since anyone can mistype a password. The evidence is in the *pattern*: many usernames, all failing, close together in time. A per-attempt classifier cannot see that pattern.

(b) Compute the four window features after each attempt until the first alert. When is the attack detected? Give the delay from the start of the episode and from the completion of its first attempt.

The window at t_i is $(t_i - 30,\ t_i]$.

t_i	window	attempts in it	count	users	fail fraction	score sum	alert?
1	$(-29, 1]$	1	1	1	0	0.05	no
13	$(-17, 13]$	1, 2	2	2	0.50	0.25	no
21	$(-9, 21]$	1–3	3	3	0.67	0.50	no
28	$(-2, 28]$	1–4	4	4	0.75	0.80	no (sum)
36	$(6, 36]$	2–5	4	4	1.00	0.97	**yes**

At $t = 28$ three of the conditions hold, but the score sum 0.80 is below 0.90: the benign attempt 1 is still in the window and dilutes it. At $t = 36$ attempt 1 has left the window, and all four conditions hold. Detection at **36 s**: $36 - 12 = 24$ s after the episode starts, and $36 - 13 = 23$ s after its first attempt completes.

(c) Repeat with a window of $W = 15$ s. Is an alert raised? Explain the benefit and the limitation of a shorter window.

Windows $(t_i - 15,\ t_i]$: at 13 they hold $\{1, 2\}$; at 21, $\{2, 3\}$; at 28, $\{3, 4\}$ (attempt 2, completed at exactly 13, is excluded); at 36, $\{4, 5\}$; at 42, $\{4, 5, 6\}$; at 51, $\{6, 7\}$. No window ever has 4 attempts, so **no alert**.

Benefit: a short window forgets old benign activity quickly and reacts fast to dense bursts. *Limitation:* it only sees attacks faster than about 4 attempts per 15 s. This attacker, at roughly one attempt every 7–8 s, is too slow for it.

(d) What is the smallest integer window length W for which the alert is raised at $t = 36$?

By $t = 36$ the completed attempts are 1–5. A window is an interval of time ending at 36, so to hold at least 4 attempts it must reach back at least to attempt 2 ($t_2 = 13$): $13 > 36 - W \iff W > 23$. With attempts 2–5 inside, all four conditions hold, by the table in (b). (A window long enough to include attempt 1 also passes, with 5 attempts, failure fraction 0.8 and sum 1.02.) The smallest integer is $W = 24$, whose window $(12, 36]$ contains exactly attempts 2–5. Check $W = 23$: the window $(13, 36]$ holds only attempts 3–5, so there is no alert at 36; the first alert then comes at 42.

(e) State which information is available at each decision time. Give two pieces of information that would introduce future-information leakage.

At time t_i the detector may use attempts completed at or before t_i: their times, usernames, outcomes and scores. Examples of leakage:

- the total number of attempts in the episode, or its end time, which are known only after it ends;
- an outcome observed later, such as "a later attempt from this source succeeded" or the incident ticket that labels the episode;
- window features computed over a centred window $(t_i - 15,\ t_i + 15]$, which uses attempts that have not happened yet.

(f) The source address belongs to the university's VPN gateway, shared by 300 students. How can this affect the window features? Suggest a better grouping key.

Every student's logins are merged into one stream. Legitimate typos by many people add failures and distinct usernames, causing false alerts. At the same time an attack is diluted by legitimate successes, lowering the failure fraction and hiding it. Better keys: the internal (pre-NAT) address or VPN session from the gateway logs, a device identifier, or at least the pair (source address, target username) to catch password guessing on one account.

(g) Consider discounted accumulation $A_i = 0.5A_{i-1} + s_i$, $A_0 = 0$, with alert threshold 0.55. Compute A_i until it first exceeds the threshold, and compare with (b).

i	1	2	3	4	5	6	7
A_i	0.050	0.225	0.363	0.481	0.461	**0.580**	–

The first value above 0.55 is $A_6 = 0.5 \times 0.4606 + 0.35 = 0.580$, at $t = 42$: six seconds *later* than the window detector. Note that $A_5 < A_4$: a weaker attempt ($s_5 = 0.22$) can *lower* the accumulated evidence, because half of the past is forgotten at each step. The discount factor and the threshold interact, and choosing them requires labelled episodes.

> **(h)** A patient attacker sends one attempt every 20 s, each with score 0.20. Show that the 30-second window detector can never alert. Show that A_i tends to $s/(1-\lambda)$ for $A_i = \lambda A_{i-1} + s$, and find the smallest λ for which the accumulator eventually alerts with threshold 0.55.

Window. Attempts are 20 s apart, so a window $(t-30,\ t]$ ending at an attempt contains that attempt and the one 20 s earlier, but not the one 40 s earlier: at most 2 attempts, never 4.

Accumulator. Unrolling, $A_i = s(1+\lambda+\cdots+\lambda^{i-1}) = s\,\frac{1-\lambda^i}{1-\lambda}$, which increases towards $s/(1-\lambda)$ for $0 \le \lambda < 1$. It eventually exceeds 0.55 if and only if $0.20/(1-\lambda) > 0.55 \iff \lambda > 1 - 0.20/0.55 = 7/11 \approx 0.636$. With $\lambda = 0.5$ the limit is 0.40, so the attacker is never caught.

Raising λ catches slow attacks but also lets ordinary low-score activity from busy legitimate hosts accumulate into false alerts. This accumulator also ignores time: discounting per attempt rather than per second treats a day of sparse failures like a burst. Real systems decay with elapsed time, for example by $e^{-\Delta t/\tau}$.

> **(i)** Describe a leakage-safe train, validation and test split when several windows are generated from each attack episode.

Split by **episode** (and preferably by **time**): all windows from one episode, and all windows from the same source during that period, go to the same subset; train on earlier periods and test on later ones. Otherwise overlapping windows of one episode, which share most of their attempts, appear in both training and test, and the test measures memory of that episode. Thresholds and window lengths are tuned on the validation episodes only.

2 Part B: Quick drills

Two minutes each, no context: the computations that Part A builds on.

> **(D1)** $C_{FP} = 5$, $C_{FN} = 20$. Give the posterior threshold and the corresponding threshold on a logistic score.

$t = 5/25 = 0.2$; on the score, $\text{logit}(0.2) = \ln(0.25) \approx -1.386$.

> **(D2)** A logistic model outputs $s = 2$ for an example with true label 0. Compute the posterior and the cross-entropy.

$\sigma(2) = 1/(1+e^{-2}) \approx 0.881$; $\ell = -\ln(1-0.881) = -\ln 0.119 \approx 2.13$. Confident and wrong is expensive.

> **(D3)** Training prior $\pi_t = 0.3$, deployment prior $\pi_d = 0.1$, calibrated $r_t(x) = 0.5$. Compute $r_d(x)$.

In odds form, $\text{odds}_d = 1 \times \frac{0.1/0.9}{0.3/0.7} = \frac{0.1111}{0.4286} \approx 0.259$, so $r_d = 0.259/1.259 \approx 0.206$.

> **(D4)** $TP = 45$, $FP = 15$, $FN = 5$, $TN = 935$. Accuracy, precision, recall, F_1?

Accuracy $980/1000 = 0.98$; precision $45/60 = 0.75$; recall $45/50 = 0.90$; $F_1 = 2(0.75)(0.9)/1.65 \approx 0.818$.

(D5) Query $(0, 0)$, training points $A = (3, 3)$ and $B = (0, 4.5)$. Which is nearest under Euclidean distance, and which under Manhattan distance?

Euclidean: $d_A = \sqrt{18} \approx 4.24 < d_B = 4.5$, so A. Manhattan: $d_A = 6 > d_B = 4.5$, so B. The choice of distance is part of the model.

(D6) One-dimensional data: 1 (benign), 2 (attack), 2.5 (attack), 4 (benign), 5 (benign). Classify the query 3.2 with 1-NN, 3-NN and 5-NN (majority vote).

Distances: 2.2, 1.2, 0.7, 0.8, 1.8. 1-NN: the point 2.5, so attack. 3-NN: 2.5 (attack), 4 (benign), 2 (attack), so attack. 5-NN: 2 attack against 3 benign, so benign. Large k drifts towards the majority class.

(D7) Two configurations with the same population error 0.18, independent validation noise $\sigma = 0.03$. Expected reported error of the winner?

$\mathbb{E}[\min] = R - \sigma/\sqrt{\pi} = 0.18 - 0.03/1.7725 \approx 0.163$ (Tutorial 1, Problem 5(d) formula).

(D8) One feature, $x \mid y = 0 \sim \mathcal{N}(0, 1)$ and $x \mid y = 1 \sim \mathcal{N}(2, 1)$, priors 0.9 and 0.1. Find the Bayes decision threshold on x.

Equal posteriors: $-\frac{x^2}{2} + \ln 0.9 = -\frac{(x-2)^2}{2} + \ln 0.1 \iff 2x - 2 = \ln 9 \iff x^\star = 1 + \frac{1}{2}\ln 9 \approx 2.10$. Predict class 1 for $x > 2.10$: the threshold sits past the class-1 mean because class 1 is rare.

(D9) The word "invoice" appears in 0 of 50 legitimate training emails. Give its MLE and Laplace-smoothed probability for the legitimate class (binary feature).

MLE $0/50 = 0$, which would veto the legitimate class for every email containing the word. Smoothed: $(0 + 1)/(50 + 2) = 1/52 \approx 0.019$.

(D10) An accumulator $A_i = 0.8A_{i-1} + 0.1$ starts at 0, with alert threshold 0.6. Does it ever alert? What discount would be needed?

The limit is $0.1/(1 - 0.8) = 0.5 < 0.6$, so it never alerts. It needs $0.1/(1 - \lambda) > 0.6 \iff \lambda > 5/6 \approx 0.833$.